

Domanda 17 Dare la definizione di $\Omega(f(n))$. Mostrare che se $f(n) = \Omega(g(n))$ e $g(n) = \Omega(h(n))$ allora $f(n) = \Omega(h(n))$.



$$\Omega(f(n)) = \{g(n) \mid \exists c > 0, \forall n \geq n_0, \exists n_0 \in \mathbb{N}$$

$$(0 \leq c g(n) \leq f(n))$$

① $g, h \rightarrow c_1, c_2 > 0, n_1, n_2 \in \mathbb{N}$
 $\searrow_{\max \dots}$

① $f(n) = \Omega(g(n)) \rightarrow (0 \leq c_1 g(n) \leq f(n))$

② $g(n) = \Omega(h(n)) \rightarrow 0 \leq c_2 h(n) \leq g(n)$

$$c_2 h(n) \leq c_2 g(n) \leq f(n) \quad (?)$$

$$n \geq \max\{n_1, n_2\}$$

$$0 \leq c_1 c_2 h(n) \leq c_1 g(n) \leq f(n)$$

$\nwarrow$

$$\Omega(h(n)) \leftarrow 0 \leq c \cdot h(n) \leq f(n)$$

Se $f(n) = \Omega(g(n))$ e $g(n) = \Omega(h(n))$ allora esistono $c_1, c_2 > 0, n_1, n_2 \in \mathbb{N}$ tali che per ogni $n \geq n_1$

$$0 \leq c_1 g(n) \leq f(n) \quad (1)$$

e per ogni $n \geq n_2$

$$0 \leq c_2 h(n) \leq g(n) \quad (2)$$

Ne consegue che per ogni $n \geq \max\{n_1, n_2\}$, moltiplicando (2) per c_1 si ha

$$0 \leq c_1 c_2 h(n) \leq c_1 g(n) \leq f(n)$$

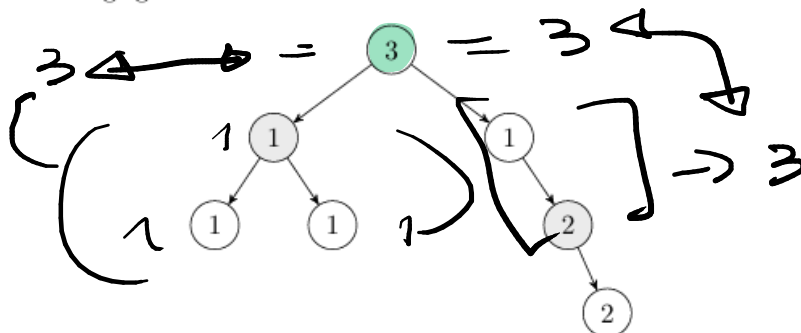
ovvero, indicato con $n_0 = \max\{n_1, n_2\}$ e $c = c_1 c_2$, per ogni $n \geq n_0$,

$$0 \leq c h(n) \leq f(n)$$

ovvero $f(n) = \Omega(h(n))$.

Esercizio 10 Un nodo x di un albero binario T si dice *fair* se la somma delle chiavi nel cammino che conduce dalla radice dell'albero al nodo x (escluso) coincide con la somma delle chiavi nel sottoalbero di radice x (con x incluso). Realizzare un algoritmo ricorsivo $printFair(T)$ che dato un albero T stampa tutti i suoi nodi fair. Supporre che ogni nodo abbia i campi $x.left$, $x.right$, $x.p$, $x.key$. Valutare la complessità dell'algoritmo.

Un esempio: i nodi grigi sono fair



// ITERATIVA

printFair(T)

x = T.root

y = x

if (x == nil) return 0 → (perché 0 == 0 allora stampiamo 0)

sum = 0

while (sum ≠ x)

if (x.left)

x = x.left

sum = sum + x.key

y = x

if (x.right)

x = x.right

sum = sum + x.key

y = x

if (sum == x.key)

printTree(y)

printTree(y)

if (T.root == nil) return x

printTree(x.left)

printTree(x.right)

// RICORSIVA

printFair (x, path) // path = somma chiavi percorso dalla radice in basso

if (x == nil) return 0

left = printFair(x.l, path + x.key)

right = printFair(x.r, path + x.key)

// albero somma

sumTree = x.key + right + left

if (path == sumTree) printTree(x)

return sumTree

Esercizio 2 (8 punti) Date due stringhe $X = \langle x_1, x_2, \dots, x_m \rangle$ e $Y = \langle y_1, y_2, \dots, y_n \rangle$, si consideri la seguente quantità $\ell(i, j)$, definita per ogni coppia di valori i, j con $0 \leq i \leq m$ e $0 \leq j \leq n$:

$$\ell(i, j) = \begin{cases} 1 & \text{se } i = 0 \text{ o } j = 0 \\ 3\ell(i, j-1) & \text{se } i, j > 0 \text{ e } x_i = y_j \\ 2\ell(i-1, j-1) - \ell(i-1, j) & \text{se } i, j > 0 \text{ e } x_i \neq y_j. \end{cases}$$

Si vuole calcolare la quantità $q = \max\{\ell(i, j) : 0 \leq i \leq m, 0 \leq j \leq n\}$.

- Scrivere un algoritmo bottom-up per il calcolo di q .
- Determinare la complessità *esatta* dell'algoritmo, supponendo che le uniche operazioni di costo unitario e non nullo siano i confronti tra caratteri.

```
COMPUTE(X, Y)
n = length(X)
m = length(Y)
max = -INFTY
```

```
if n = 0 || m == 0 then return 1
```

```
for i = 0 to m
  C[i, m] = 1
  max = MAX(max, C[i, m])
```

```
for j = 0 to n
  C[0, n] = 1
  max = MAX(max, C[0, n])
```

```
for i = 0 to m - 1
  for j = n - 1 downto 0
    if (X[i] == Y[j])
      C[i, j] = 3 * C[i, j-1]
    if (X[i] != Y[j])
      C[i, j] = 2 * C[i-1, j-1] - C[i-1, j]
  max = MAX(max, C[i, j])
```

```
return max
```

$$\ell(i, j) = \begin{cases} 1 & \text{se } i = 0 \text{ o } j = 0 \\ 3\ell(i, j-1) & \text{se } i, j > 0 \text{ e } x_i = y_j \\ 2\ell(i-1, j-1) - \ell(i-1, j) & \text{se } i, j > 0 \text{ e } x_i \neq y_j. \end{cases}$$

$$T(m) = \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} 1 = \sum_{i=0}^{m-1} (n-1) = \sum_{k=0}^{m-1} k$$

$$\rightarrow \frac{(m-1)(m-1)}{2}$$

$$\sum_{k=0}^m k = \frac{n(n-1)}{2}$$

$$\sum_{k=1}^{m-2} k = \frac{(n-2)(n-1)}{2}$$

Esercizio 2 (9 punti) Lungo una strada ci sono, in vari punti, n parcheggi liberi e n auto. Un posteggiatore ha il compito di parcheggiare tutte le auto, e lo vuole fare minimizzando lo spostamento totale da fare. Formalmente, dati n valori reali $p_1, p_2, \dots, p_n$ e altri n valori reali $a_1, a_2, \dots, a_n$, che rappresentano

$p \sim A$
 $\sim$
 $10 \quad 12$

le posizioni lungo la strada rispettivamente di parcheggi e auto, si richiede di assegnare ad ogni auto a_i un parcheggio $p_{h(i)}$ minimizzando la quantità

$$\sum_{i=1}^n |a_i - p_{h(i)}|.$$

L'idea di questo algoritmo, se fosse in codice, sarebbe simile a Metric Matching, quindi cerco di minimizzare la differenza partendo dalla fine (quindi, vedendo quante auto e quanti parcheggi ci sono) e verificando quale, a coppie, ha la minima differenza.

1. Si consideri il seguente algoritmo greedy. Si individui la coppia (auto, parcheggio) con la minima differenza. Si assegni quell'auto a quel parcheggio. Si ripeta con le auto e i parcheggi restanti fino a quando tutte le auto sono parcheggiate. Dimostrare che questo algoritmo non è corretto, esibendo un controesempio.
2. Si consideri il seguente algoritmo greedy. Si assuma che i valori $p_1, p_2, \dots, p_n$ e $a_1, a_2, \dots, a_n$ siano ordinati in modo non decrescente. Si produca l'assegnazione $(a_1, p_1), (a_2, p_2), \dots, (a_n, p_n)$. Dimostrare la correttezza di questo algoritmo per il caso $n = 2$.

Questo significa che l'assegnazione parte dagli ultimi parcheggi e dalle ultime auto (quindi, massima somma al contrario significa minima differenza).

$$p_1 = 5 \quad p_2 = 10 \quad a_1 = 12 \quad a_2 = 14$$

$$a_1 - p_1 = 12 - 5 = 7$$

$$a_2 - p_2 = 14 - 10 = 4 \leftarrow \text{min!}$$

$$\text{GREEDY} \rightarrow (4) + (7) = 11$$

$\hookrightarrow$ CONTROESEMPLO

$$\begin{array}{ccc} a_1 & \dots & a_n \\ \swarrow & & \searrow \\ p_1 & \dots & p_n \end{array}$$

$$\left(\begin{array}{cc} p_1 = 5 & p_2 = 10 \\ \swarrow & \searrow \\ a_1 = 3 & a_2 = 14 \end{array} \right)$$

$$\left(\begin{array}{cc} 10 - 5 = 5 & \\ 14 - 5 = 9 & \end{array} \right) \xrightarrow{10} \text{min} \rightarrow \text{BAD!}$$

$$\rightarrow \left(\begin{array}{cc} p_1 = 5 & p_2 = 10 \\ \swarrow & \searrow \\ a_1 = 3 & a_2 = 14 \end{array} \right)$$

$$\left(\begin{array}{cc} 3 - 5 = 2 & \\ 14 - 10 = 4 & \end{array} \right) = (8)$$

$\downarrow$ ASSEGNAZIONE POSIZIONATA

$$\left[\begin{array}{ccc} p_1 - a_1 & p_2 - a_2 & p_3 - a_3 \end{array} \right]$$

$$\sum_{i=1}^n |a_i - p_i|$$

$\downarrow$ POSIZIONATA

$$10 \quad \dots \quad 12$$

$$12 - 10 = 2 \text{ (KM } \approx \text{ DISTANZA)}$$

2. Ci sono vari casi possibili:

Dal ragionamento detto, matematicamente, si vede che basta prendere un qualsiasi ordinamento tra le due auto e i due parcheggi di due generiche differenze e si esprime la somma in termini matematici (l'idea concreta è quella spiegata da me).

(a) Caso $a_1 \leq p_1 \leq p_2 \leq a_2$

- l'assegnazione $(a_1, p_1), (a_2, p_2)$ ha costo $p_1 - a_1 + a_2 - p_2 = (a_2 - a_1) - (p_2 - p_1)$
- l'assegnazione $(a_1, p_2), (a_2, p_1)$ ha costo $p_2 - a_1 + a_2 - p_1 = (a_2 - a_1) + (p_2 - p_1)$; siccome $p_2 - p_1 \geq 0$, questa assegnazione ha costo non inferiore rispetto alla precedente

(b) Caso $a_1 \leq p_1 \leq a_2 \leq p_2$

- l'assegnazione $(a_1, p_1), (a_2, p_2)$ ha costo $p_1 - a_1 + p_2 - a_2 = (p_2 - a_1) - (a_2 - p_1)$
- l'assegnazione $(a_1, p_2), (a_2, p_1)$ ha costo $p_2 - a_1 + a_2 - p_1 = (p_2 - a_1) + (a_2 - p_1)$; siccome $a_2 - p_1 \geq 0$, questa assegnazione ha costo non inferiore rispetto alla precedente

(c) Caso $a_1 \leq a_2 \leq p_1 \leq p_2$

- l'assegnazione $(a_1, p_1), (a_2, p_2)$ ha costo $p_1 - a_1 + p_2 - a_2 = (p_2 - a_1) + (p_1 - a_2)$
- l'assegnazione $(a_1, p_2), (a_2, p_1)$ ha costo $p_2 - a_1 + p_1 - a_2 = (p_2 - a_1) + (p_1 - a_2)$, uguale a quello precedente

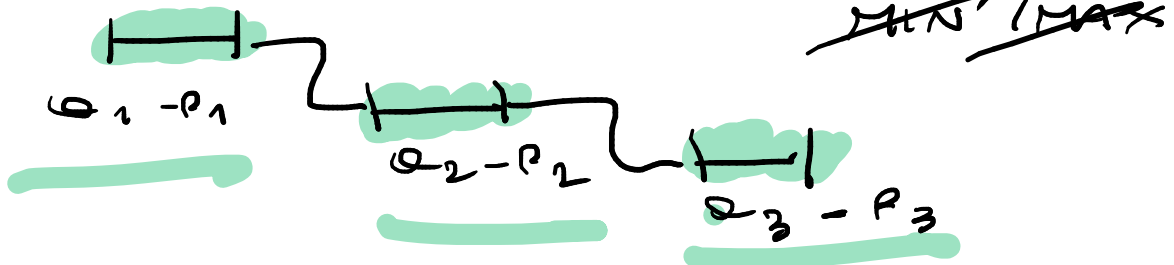
$a_i - p_i$

Tutti gli altri casi sono simmetrici e si dimostrano nella stessa maniera.

1. Si consideri il seguente input:

$$p_1 = 5, p_2 = 10 \quad \text{e} \quad a_1 = 9, a_2 = 14.$$

L'algoritmo produce l'assegnazione $(a_1, p_2), (a_2, p_1)$, che ha costo $1 + 9 = 10$, mentre l'assegnazione $(a_1, p_1), (a_2, p_2)$ ha costo $4 + 4 = 8$.



Esercizio 2 (8 punti) Per $n > 0$, siano dati due vettori a componenti intere $\mathbf{a}, \mathbf{b} \in \mathbb{Z}^n$. Si consideri la quantità $c(i, j)$, con $0 \leq i \leq j \leq n - 1$, definita come segue:

$$c(i, j) = \begin{cases} a_i & \text{se } 0 < i \leq n - 1 \text{ e } j = n - 1, \\ b_j & \text{se } i = 0 \text{ e } 0 \leq j \leq n - 1, \\ c(i - 1, j) \cdot c(i, j + 1) & 0 < i \leq j < n - 1. \end{cases}$$

Si vuole calcolare la quantità $M = \max\{c(i, j) : 0 \leq i \leq j \leq n - 1\}$.

$M(n, 1)$

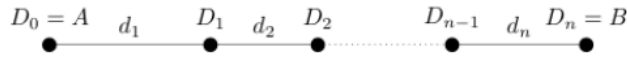
1. Si fornisca il codice di un algoritmo iterativo bottom-up per il calcolo di M .

2. Si valuti la complessità esatta dell'algoritmo, associando costo unitario ai prodotti tra numeri interi e costo nullo a tutte le altre operazioni.

Domanda B (7 punti) Si consideri il problema di selezione di attività compatibili:

1. Definire il problema.
2. Descrivere brevemente l'algoritmo ottimo GREEDY_SEL visto in classe.
3. Fornire un esempio di algoritmo greedy *non* ottimo, motivandone la non ottimalità.

Esercizio 33 Si supponga di voler viaggiare dalla città A alla città B con un'auto che ha un'autonomia pari a d km. Lungo il percorso si trovano $n - 1$ distributori $D_1, \dots, D_{n-1}$, a distanze di $d_1, \dots, d_n$ km ($d_i \leq d$) come indicato in figura



L'auto ha inizialmente il serbatoio pieno e l'obiettivo è quello di percorrere il viaggio da A a B , minimizzando il numero di soste ai distributori per il rifornimento.

- Introdurre la nozione di soluzione per il problema e di costo della una soluzione. Mostrare che vale la proprietà della sottostruttura ottima e individuare una scelta che gode della proprietà della scelta greedy.
- Sulla base della scelta greedy individuata al passo precedente, fornire un algoritmo greedy $\text{stop}(d, n)$ che dato in input l'array delle distanze $d[1..n]$ restituisce una soluzione ottima.
- Valutare la complessità dell'algoritmo.

(*)

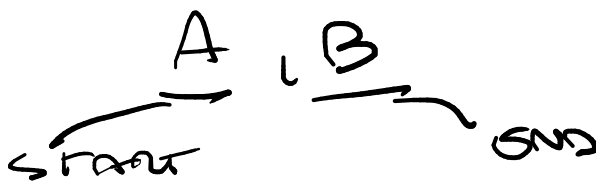
Am B

$$\min \sum_{i=1}^m d_i = \text{THRESHOLD} = T = 300 \text{ KM}$$

$m = 3$

(*) $\rightarrow \sum D_i = \text{DISTRIBUTORI} \rightarrow n-1$

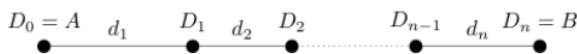
$\sum d_i = \text{DISTANZE}$



$$\left[\sum_{i=1}^m d_i \leq T \right]$$

AUTONOMIA
NON
FINISCO

Esercizio 33 Si supponga di voler viaggiare dalla città A alla città B con un'auto che ha un'autonomia pari a d km. Lungo il percorso si trovano $n - 1$ distributori $D_1, \dots, D_{n-1}$, a distanze di $d_1, \dots, d_n$ km ($d_i \leq d$) come indicato in figura



L'auto ha inizialmente il serbatoio pieno e l'obiettivo è quello di percorrere il viaggio da A a B , minimizzando il numero di soste ai distributori per il rifornimento.

- Introdurre la nozione di soluzione per il problema e di costo della una soluzione. Mostrare che vale la proprietà della sottostruttura ottima e individuare una scelta che gode della proprietà della scelta greedy.
- Sulla base della scelta greedy individuata al passo precedente, fornire un algoritmo greedy $\text{stop}(d, n)$ che dato in input l'array delle distanze $d[1..n]$ restituisce una soluzione ottima.
- Valutare la complessità dell'algoritmo.

$$(\text{DIST} - \text{AUTONOMIA}) > 0 ?$$



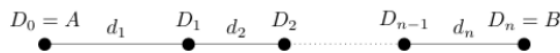
$i = \max \{ j \mid D_j \leq d \}$ $\rightarrow$ DIST + d[i] > d[i-1]

$\rightarrow$ MAX FINISCO

Scelta greedy. Per il problema $D_0, D_1, \dots, D_n$, si fissa necessariamente $i_0 = 0$, e la scelta greedy consiste nel raggiungere la sosta più lontana a distanza minore o uguale di d , ovvero definire $i_1 = \max\{j \mid d_{0,j} \leq d\}$.

Data una soluzione ottima per il problema $S = D_{j_0} \dots D_{j_k}$, certamente $j_0 = 0 = i_0$ e $j_1 \leq i_1$. Quindi è immediato verificare che anche $S' = D_{j_0} D_{i_1} D_{j_2} \dots D_{j_k}$ è una soluzione per il problema $D_0, D_1, \dots, D_n$, ed è ottima, dato che $c(S') = c(S)$. (Si noti che, più precisamente, in prima istanza si potrebbe pensare che D_{i_1} potesse essere anche oltre D_{j_2} , ma questo darebbe una soluzione migliore di quella ottima, portando ad un assurdo).

Esercizio 33 Si supponga di voler viaggiare dalla città A alla città B con un'auto che ha un'autonomia pari a d km. Lungo il percorso si trovano $n - 1$ distributori $D_1, \dots, D_{n-1}$, a distanze di $d_1, \dots, d_n$ km ($d_i \leq d$) come indicato in figura



L'auto ha inizialmente il serbatoio pieno e l'obiettivo è quello di percorrere il viaggio da A a B , minimizzando il numero di soste ai distributori per il rifornimento.

i. Introdurre la nozione di soluzione per il problema e di costo della una soluzione. Mostrare che vale la proprietà della sottostruttura ottima e individuare una scelta che gode della proprietà della scelta greedy.

ii. Sulla base della scelta greedy individuata al passo precedente, fornire un algoritmo greedy $\text{stop}(d, n)$ che dato in input l'array delle distanze $d[1..n]$ restituisce una soluzione ottima.

iii. Valutare la complessità dell'algoritmo.

$\text{stop}(d, n)$

$\text{dist} = d[1]$
 $\text{last} = 1$

for $i = 2$ to n
 if $d[i] + \text{dist} \geq d[\text{last}]$
 $\text{dist} = d[i]$
 $\text{last} = i$

return dist

```

STOP(d, n)
1. pos = 0 // posizione corrente (partiamo da A = D_0)
2. soste = [] // array delle soste effettuate
3. while pos < n // finché non abbiamo raggiunto B
4.     next = pos
5.     // Trova il distributore più lontano raggiungibile
6.     for j = pos + 1 to n
7.         if d[j] - d[pos] ≤ d then
8.             next = j
9.         else
10.            break // superata l'autonomia
11.     if next == pos then
12.         return IMPOSSIBILE // B non raggiungibile
13.     if next < n then // se non siamo già a B
14.         APPEND(soste, next)
15.     pos = next
16. return soste

```

Enunciato: Sia S^* una soluzione ottima per il problema su tutto il percorso da A a B . Sia D_k l'ultimo distributore in S^* prima di B . Allora $S^* \setminus \{D_k\}$ è una soluzione ottima per il sottoproblema da A a D_k .

Dimostrazione (per assurdo):

Supponiamo che esista una soluzione S' per il sottoproblema $A \rightarrow D_k$ con costo strettamente minore: $|S'| < |S^* \setminus \{D_k\}|$.

Allora la soluzione $S' \cup \{D_k\}$ avrebbe costo $|S'| + 1 < |S^* \setminus \{D_k\}| + 1 = |S^*|$, contraddicendo l'ottimalità di S^* .

Proprietà della scelta greedy:

– Esiste sempre una soluzione ottima che include il distributore più lontano raggiungibile da A (o dall'ultima posizione corrente).

– Intuizione: Posticipare il più possibile ogni rifornimento minimizza il numero totale di soste necessarie.

Scelta greedy formale: Da una posizione corrente p con autonomia residua d km, scegli il distributore D_j tale che:

- $d_j \leq p + d$ (raggiungibile)
- d_j è massimo tra tutti i distributori raggiungibili

Algoritmo. Ne segue l'algoritmo che riceve in la sequenza di distanze nella forma di un array $d[1, n]$ e restituisce un array $S[1..n - 1]$ con le soste scelte (la prima e l'ultima sono scelte sempre, non serve indicarlo).

```
stop(d, n)
  dist = d[1]                // distanza percorsa
  for i=2 to n
    if dist + d[i] > d
      S[i-1]=1
      dist=d[i]
    else
      S[i-1] = 0
  return S
```

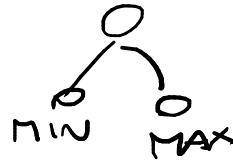


Per scelta greedy, prendiamo la sosta con distanza $\leq$ a quella attuale, tale da capire caso per caso quale conviene di più.

L'algoritmo ricalca proprio la selezione delle attività e:
 - prende come distanza la prima, per inizializzazione
 - se la somma tra la prima distanza e quella attuale è $>$ della sequenza delle distanze, allora abbiamo la distanza buona e salviamo la sosta con la distanza ottima (sapendo che ci siamo fermati "prima", quindi $S[i-1]=1$)
 - se invece non ci siamo fermati prima, avremo $S[i-1] = 0$

La complessità è $\Theta(n)$.

Domanda 30 Scrivere una funzione $\text{diff}(T)$ che dato in input un albero binario T determina la massima differenza di lunghezza tra due cammini che vanno dalla radice ad un sottoalbero vuoto. Ad esempio sull'albero ottenuto inserendo 1, 2 e 3 produce 2, su quello ottenuto inserendo 2, 1, 3 produce 0. Valutarne la complessità.

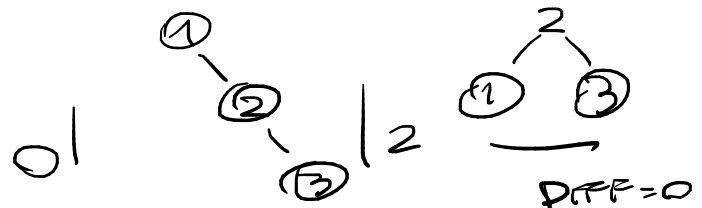


```
diff(T)
  min, max = diffRec(T.root)
  return
```

```
diffRec(x)
// range
```

```
  minL, maxL = diffRec(x.left, level_l + 1)
  maxR, maxR = diffRec(x.right, level_r + 1)
```

```
  if(level_l > level_r)
    return max(maxL, maxR) + 1
  else
    return min(minL, maxR) + 1
```



DIFF = 2

```
diff(T)
1. if T.root == nil then
2.   return 0
3. _, maxDiff = diffRec(T.root)
4. return maxDiff
```

```
diffRec(x)
1. if x == nil then
2.   return -1, 0 // altezza = -1, diff = 0
3.
4. // Ricorsione sui sottoalberi
5. hL, diffL = diffRec(x.left)
6. hR, diffR = diffRec(x.right)
7.
8. // Altezza corrente
9. h = max(hL, hR) + 1
10.
11. // Differenza locale in questo nodo
12. localDiff = |hL - hR|
13.
14. // Massima differenza trovata nel sottoalbero radicato in x
15. maxDiff = max(localDiff, diffL, diffR)
16.
17. return h, maxDiff
```

Esercizio: matching sulla linea

Sia $S = \{s_1, s_2, \dots, s_n\}$ un insieme di punti ordinati sulla retta reale, rappresentanti dei server. Sia $C = \{c_1, c_2, \dots, c_n\}$ un insieme di punti ordinati sulla retta reale, rappresentanti dei client. Il costo di assegnare un client c_i ad un server s_j è $|c_i - s_j|$. Fornire un algoritmo greedy che assegna ogni client ad un server distinto e che minimizzi il costo totale (equiv. medio) dell'assegnamento.

$$c_i - s_j$$

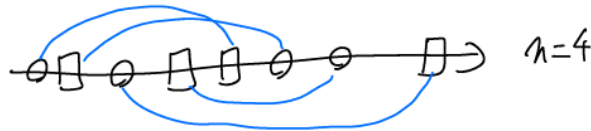
$$c_n - s_n$$

$$c_2 - s_2$$

$$\min \sum$$

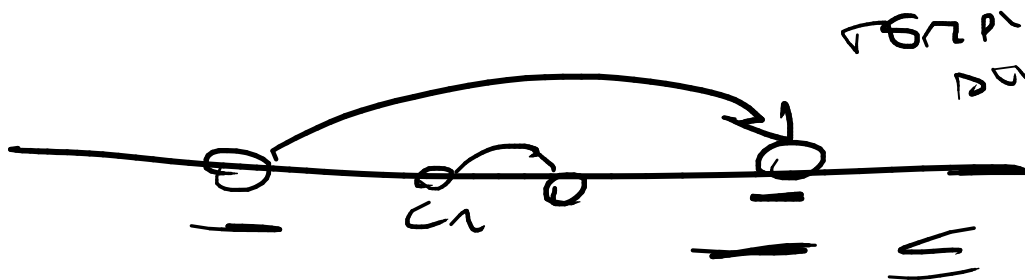
$$c_i - s_i$$

esempio:



CONTRAESORPLE $\rightarrow \min (??) \rightarrow \underline{\underline{NO!}}$

$$c_1 + c_2 + \dots + c_n \leq n \rightarrow \underline{\underline{NO}}$$



ESERPI
DOWNSC $\rightarrow$ LI

$$\min \left(\begin{matrix} \circ & \circ \\ \circ & \circ \end{matrix} \right) \rightarrow$$

ASSIGN. PICCOLI

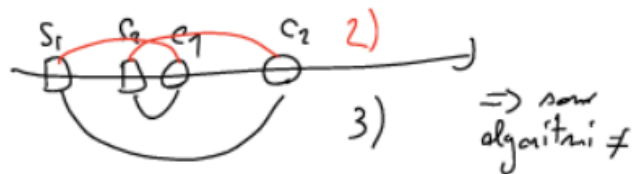
PUNZONA

~~1)~~ client & server + vicini, partendo dalla coppia client-server con distanza minore

NON funziona:  OPT

2) $C_1 - S_1, C_2 - S_2, \dots$
(equiv. $C_n - S_n, C_{n+1} - S_{n+1}, \dots$)

~~3)~~ C_1 al server + vicini; C_2 al server + vicini



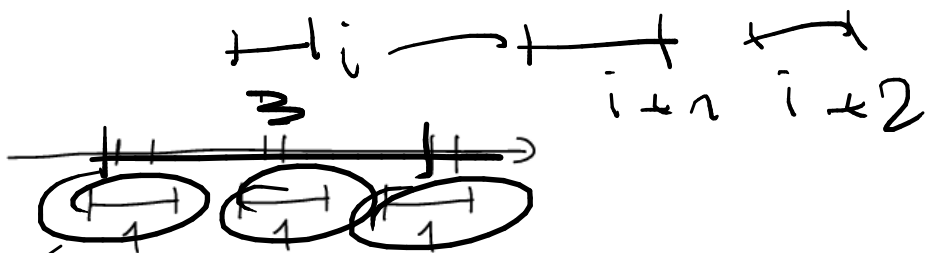
NON funziona:



Esercizio: Sia $X = \{x_1, x_2, \dots, x_n\}$ un insieme di punti ordinati sulla retta reale.

Fornire un algoritmo greedy che determini un insieme I di cardinalità minima di intervalli chiusi di ampiezza unitaria ($[a, b] \in I \Rightarrow \underline{b-a=1}$) tale che $\forall x_i \in X \exists j \in I$ tale che $\underline{x_i \in j}$.

esempio:



MIN_COVER(X)
 $n = \text{length}(X)$
 $C = \{ [x_1, x_1+1] \}$
 $\text{last} = 1$

for $i=2$ to n do
 if $\left\{ \begin{array}{l} x_i > x_{\text{last}} + 1 \end{array} \right\}$ then
 $C = C \cup \{ [x_i, x_i+1] \}$
 $\text{last} = i$
 return C

COPERTURA MINIMA $\rightarrow$
 TANTI VALORI
 PICCOLI

(...)

Domanda 10 Si consideri la ricorrenza

$$T(n) = T\left(\frac{4n}{5}\right) + \frac{n}{2} + \log n$$

Fornire limite asintotico stretto per la soluzione.

Caso 3: $f(n) \rightarrow \log(n)$ vs $n \log_b(a) \rightarrow 1$
Deve quindi valere la condizione di regolarità, risolvendo per k .
Quindi, il limite stretto è per $\theta(n \log_b(a)) \rightarrow \theta(n)$

INDUZIONE FORTE

$$n > n^2$$

$$T(n) \leq C(n)$$

$$T(n-1) \leq C(n-1)$$

↑

RAFFORZA

IPOTESI

INDUTTIVA